

# TP — Reverser un binaire anti-debug

**En une phrase** : un programme refuse de dévoiler son *flag* quand il est lancé sous un debugger. Votre travail : comprendre **comment** il détecte le debugger, puis le **tromper** pour récupérer le flag.

## Le but du TP (lisez ceci en premier)

Le binaire `protected_target` contient **4 pièges** qui repèrent la présence d'un debugger (GDB). Si **un seul** piège se déclenche, le programme s'arrête et vous n'obtenez **rien**.

Votre mission se résume à **3 objectifs**, du plus simple au plus avancé :

| # | Objectif                      | Ce que vous devez rendre / montrer                                    |
|---|-------------------------------|-----------------------------------------------------------------------|
| A | Comprendre les 4 protections  | Une phrase par protection : <i>comment</i> elle détecte le debugger   |
| B | Contourner à la main dans GDB | Le flag affiché à l'écran + les commandes utilisées                   |
| C | Automatiser le contournement  | Un script <code>bypass.gdb</code> complété qui sort le flag tout seul |

 **Vous avez gagné quand vous voyez :**

 SECRET FLAG: FLAG{...}

 Le flag n'apparaît **jamais** en exécution normale. Il ne s'affiche **que** si vous réussissez à faire tourner le programme **sous debugger sans vous faire détecter**. C'est là toute la difficulté.

## Prérequis (5 min)

```
sudo apt-get update && sudo apt-get install -y gdb build-essential
gdb --version          # doit répondre une version
```

Connaissances utiles (pas besoin d'être expert) :

- Lire quelques lignes d'assembleur x86-64.
- Savoir qu'une fonction C **renvoie sa valeur dans le registre `rax`**.
- Convention du TP : **chaque fonction `check_*` renvoie `0` = OK, `1` = debugger détecté**.

## Ce qu'il y a dans le dossier

```
src/
protected_target.c  ← le code source (LISEZ-LE, tout est dedans)
```

|                   |                                                  |
|-------------------|--------------------------------------------------|
| Makefile          | ← compile le binaire                             |
| exploits/         |                                                  |
| bypass.gdb        | ← script GDB à COMPLÉTER (objectif C)            |
| anti_debug_hook.c | ← bonus : hook LD_PRELOAD (fourni, à comprendre) |
| patch_binary.py   | ← bonus avancé : patch du binaire                |
| TEST.sh           | ← vérifie automatiquement que tout marche        |

📌 **Conseil clé** : ouvrez `src/protected_target.c` en même temps que GDB. L'assembleur devient 10× plus clair quand on a le C sous les yeux.

---

## ÉTAPE 0 — Compiler et observer (10 min)

---

### 1. Compiler

```
cd src/  
make
```

Vous devez voir ✅ **Compilation réussie** !.

### 2. Voir le comportement NORMAL

```
./protected_target
```

👉 Le programme se lance, passe ses checks, mais **cache le flag** (message « flag secret non accessible »).

### 3. Voir la DÉTECTION sous GDB

```
gdb ./protected_target  
(gdb) run
```

👉 Le programme **détecte GDB dès le 1er check** et s'arrête : ❌ **DEBUGGER DÉTECTÉ (ptrace)** !

➡ **Le paradoxe à résoudre** : en normal, pas de flag. Sous GDB, détection. Il faut donc **lancer sous GDB et neutraliser les 4 checks**.

---

## ÉTAPE A — Comprendre les 4 protections (le cœur du TP)

---

Listez d'abord les fonctions à analyser :

```
nm protected_target | grep check_
```

→ `check_ptrace`, `check_proc_status`, `check_timing`, `check_breakpoints`.

Pour chacune, faites la même chose : **lire le C**, puis **regarder l'assembleur**.

```
gdb ./protected_target
(gdb) disassemble check_ptrace
# (tapez `c` si GDB affiche "Type <RET> for more...")
```

Pour **chaque** protection, répondez à 3 questions (c'est votre rendu de l'objectif A) :

1. **Quelle astuce** utilise-t-elle pour repérer le debugger ?
2. **Quelle valeur** trahit la présence du debugger ?
3. **Que renvoie** la fonction dans ce cas ? (rappel : **1** = détecté)

---

## 1 `check_ptrace` — l'appel système ptrace

**Idée** : sous Linux, un processus ne peut être tracé **qu'une seule fois**. `ptrace(PT_TRACE_ME, ...)` réussit si personne ne trace, mais **échoue** (**< 0**) si GDB est déjà attaché.

- 🔍 Cherchez le `call ptrace` dans le désassemblage.
- Puis le `test %rax,%rax` (teste le résultat) suivi d'un saut conditionnel.
- **Aide lecture d'asm** :
  - `rax` = valeur de retour du dernier `call`.
  - `test %rax,%rax` puis `jns <adr>` : « saute si le résultat est  $\geq 0$  » (donc « saute si ptrace a réussi = pas de debugger »).
- 👉 Si le saut **n'est pas** pris, le code enchaîne sur la branche « détecté » qui charge **1** dans `eax`.

---

## 2 `check_proc_status` — le fichier /proc

**Idée** : le noyau écrit le PID du traceur dans `/proc/self/status`, ligne `TracerPid: N`. Si `N != 0`, quelqu'un trace le processus.

- La fonction ouvre le fichier avec `fopen("/proc/self/status", "r")` puis le lit ligne par ligne (`fgets`) en cherchant `TracerPid:`.
- 👉 Détail malin : elle vérifie ensuite le **nom** du traceur dans `/proc/<pid>/comm`. Elle ne renvoie **1** que si ce nom contient `gdb`, `strace`, `lldb` ou `radare`.

---

## 3 `check_timing` — l'attaque temporelle

**Idée** : exécuter pas-à-pas (single-step) sous debugger est **beaucoup plus lent**.

- La fonction mesure le temps d'une boucle avec `clock_gettime(CLOCK_MONOTONIC, ...)`.

- **Seuil de détection : 50 000 µs (50 ms).** Au-delà → **✗ debugger**.
- **👉** En pratique, ce check ne se déclenche que si vous *stepperez* la boucle ; en exécution *continue* normale il passe. Mais votre bypass doit quand même le neutraliser par sécurité.

---

#### 4 `check_breakpoints` — le scan de 0xCC

**Idée :** un breakpoint logiciel GDB remplace le **1er octet** d'une instruction par **0xCC** (l'instruction **INT3**). La fonction **se scanne elle-même** pour trouver ce **0xCC**.

- Elle prend `func_ptr = (unsigned char*)check_breakpoints` et lit ses **32 premiers octets** ; si l'un vaut **0xCC**, un breakpoint est posé → renvoie **1**.
- **👉** Conséquence pratique : **ne mettez PAS de breakpoint directement sur `check_breakpoints`**, vous vous feriez détecter par vous-même. On la contourne autrement (voir Étape B).

**Rappel du mécanisme d'un breakpoint GDB :**

1. GDB sauvegarde l'octet original (**0x55** = `push %rbp`).
2. Il écrit **0xCC** à la place.
3. Le CPU exécute **0xCC** → interruption **SIGTRAP**.
4. GDB attrape le signal et stoppe le programme.
5. Au *continue*, GDB restaure temporairement l'octet original.

---

## ÉTAPE B — Contourner à la main dans GDB (objectif principal)

---

**Stratégie :** on laisse chaque `check_*` s'exécuter, puis on **force sa valeur de retour à 0** avant que `main` ne la teste. Le programme croit alors que tout va bien.

Recette pas-à-pas

```
gdb ./protected_target
```

```
# 1. Poser un breakpoint sur chaque protection D'UN COUP
(gdb) break check_ptrace
(gdb) break check_proc_status
(gdb) break check_timing
(gdb) break check_breakpoints
(gdb) run
```

À **chaque** arrêt, répétez ce trio :

```
(gdb) finish                # laisse la fonction finir ; GDB affiche "Value
returned = 1"
(gdb) set $rax = 0          # ★ on écrase le "1" (détecté) par "0" (OK)
(gdb) continue              # on repart jusqu'au check suivant
```

★ **La commande à retenir** : `set $rax = 0`. C'est elle qui transforme « debugger détecté » en « tout va bien ».

Et le flag ?

En exécution normale, `main` n'appelle jamais `show_secret()`. Une fois les 4 checks passés, appelez la fonction secrète vous-même :

```
(gdb) call show_secret()
```

✓ **Résultat attendu** :

```
🚩 SECRET FLAG: FLAG{N0_D3bugg3r_D3t3ct3d}
```

⚠ **Rappel Étape 4** : le breakpoint sur `check_breakpoints` ne vous détecte pas ici, car GDB retire son `0xCC` juste avant d'exécuter la fonction (au moment du `continue`). Le scan ne voit donc que l'octet original. 👍

---

## ÉTAPE C — Automatiser avec un script GDB

---

Ouvrez `exploits/bypass.gdb`. Le principe : GDB permet d'attacher des commandes à un breakpoint (bloc `commands ... end`) pour rejouer automatiquement le trio de l'Étape B.

Squelette d'un bloc à compléter :

```
break check_ptrace
commands
  silent
  return 0          # ★ force le retour à 0 dès l'entrée de la fonction
  continue
end
```

**Votre travail** :

- Écrire un bloc identique pour les 4 fonctions `check_*`.
- Ajouter un bloc sur `normal_execution` qui appelle `call show_secret()` pour afficher le flag.

Lancer le script :

```
gdb -x ../exploits/bypass.gdb ./protected_target
```

✅ **Résultat attendu** : le flag s'affiche **tout seul**, sans aucune interaction.

💡 Deux façons de forcer le retour, les deux marchent :

- `return 0` à l'entrée de la fonction (elle ne s'exécute même pas).
- ou `finish + set $rax = 0` (comme en manuel).

---

## ✅ Vérifier votre travail automatiquement

---

Un script teste les 5 étapes (compilation, détection, bypass, hook) :

```
./TEST.sh
```

Il doit finir par afficher le flag sur la ligne ✅ `Bypass OK - Flag: FLAG{...}`.

---



## BONUS (optionnel)

---

Bonus 1 — Hook avec LD\_PRELOAD (sans toucher au binaire ni à GDB)

On remplace les vraies fonctions système par les nôtres au chargement.

```
cd exploits/  
gcc -shared -fPIC -o anti_debug_hook.so anti_debug_hook.c -ldl  
LD_PRELOAD=./anti_debug_hook.so ../src/protected_target
```

Le code fourni (`anti_debug_hook.c`) intercepte :

- `ptrace()` → renvoie toujours 0,
- `fopen("/proc/self/status")` → renvoie un faux fichier avec `TracerPid: 0`,
- `clock_gettime()` → maintient un delta minuscule.

👉 **À comprendre** : pourquoi ce hook **ne neutralise pas** `check_breakpoints` ? (Indice : ce check ne passe par aucune fonction de la libc.)

Bonus 2 — Patcher le binaire définitivement

```
cd exploits/  
python3 patch_binary.py ../src/protected_target  
../src/protected_target.patched  
../../src/protected_target.patched
```

Le script cherche les instructions de test/saut (`test eax, eax, cmp al, 0xCC`, le seuil de timing) et les remplace par des `NOP` ou modifie le seuil. **Avancé** : demande de comprendre l'encodage des instructions x86-64.

---

## Antisèche GDB

```
disassemble <fonction>      # voir l'assembleur d'une fonction  
info functions              # lister les fonctions  
break <fonction>            # breakpoint sur une fonction  
run / continue              # lancer / reprendre  
finish                      # sortir de la fonction courante  
print $rax                  # lire un registre  
set $rax = 0                # ★ écrire dans un registre  
call show_secret()          # appeler une fonction du binaire  
set pagination off          # éviter les "--More--"
```

---

## Checklist finale

- ☐ **A.** J'ai décrit, en 1 phrase chacune, les 4 protections.
- ☐ **B.** J'ai obtenu le flag à la main dans GDB (`finish + set $rax=0 + call show_secret()`).
- ☐ **C.** Mon `bypass.gdb` complété affiche le flag tout seul.
- ☐ `./TEST.sh` se termine avec `Flag: FLAG{...}`.
- ☐ (Bonus) J'ai fait tourner le hook `LD_PRELOAD` et/ou le patch Python.

**Bon reverse !** 